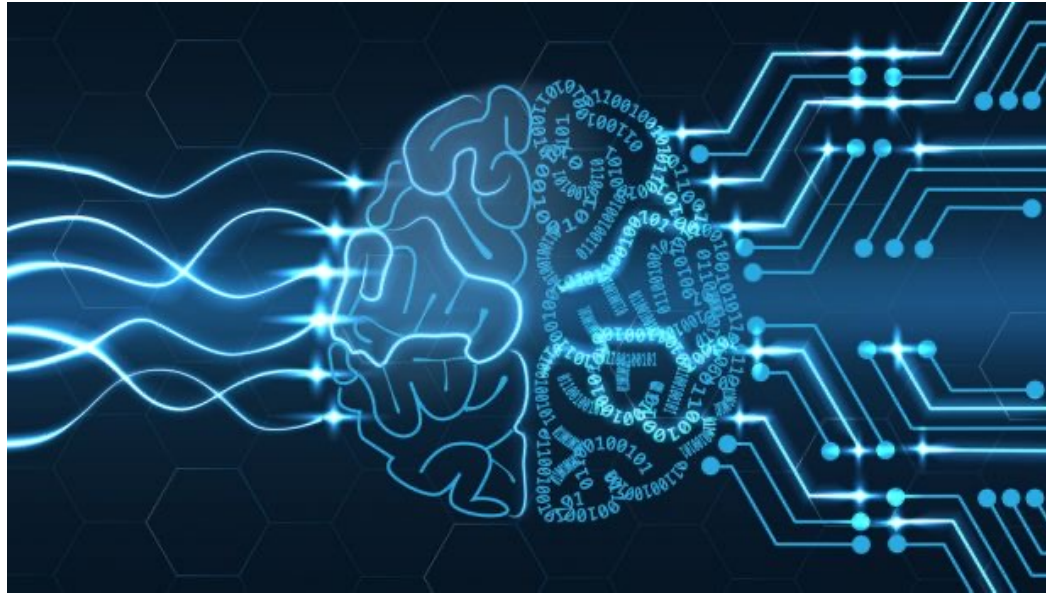


Online Student Experience Questionnaire

March 25th – April 8th

Check your Carleton e-mail for a
personalized link



Chapter 7: Unsupervised Learning

Changcheng Huang

Department of Systems and
Computer Engineering

Outline

- Unsupervised Learning Basics
- Structured Probabilistic Model
- Training for Undirected Graphical Models
- Restricted Boltzmann Machine

Outline

- Unsupervised Learning Basics
- Structured Probabilistic Model
- Training for Undirected Graphical Models
- Restricted Boltzmann Machine

What is unsupervised learning?

- Given $\mathcal{T} = \{\mathbf{x}^{(i)}, i = 1, \dots, M\}$
 - Find $p(X)$ or its properties (or patterns)
 - Generate examples from $p(X)$

- Converting to supervised learning

$$p(X_1, X_2, \dots, X_N) = p(X_1) \prod_2^N p(X_i | X_1, \dots, X_{i-1})$$

- High dimensionality

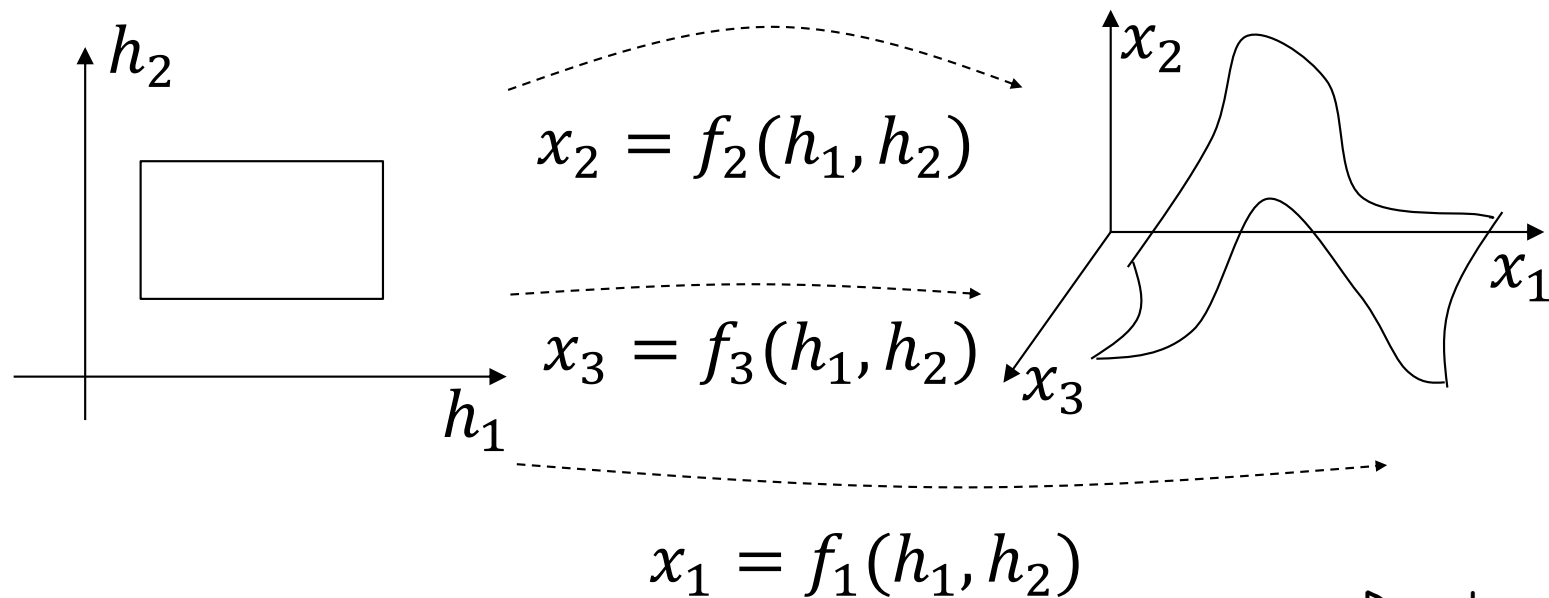
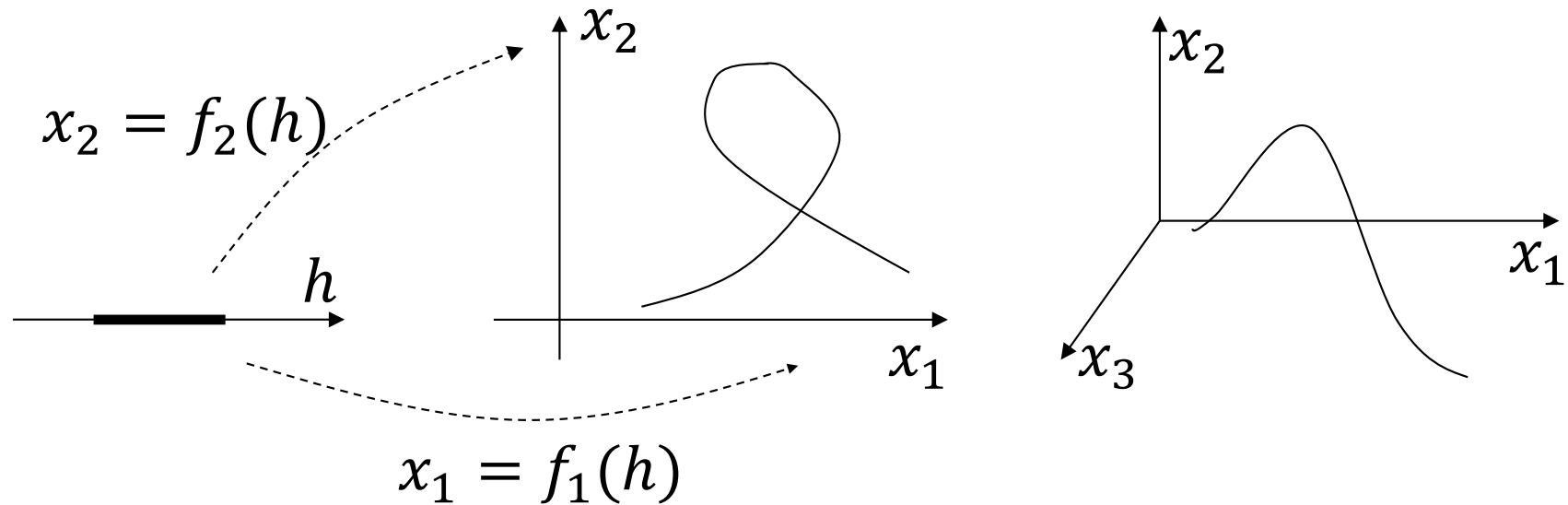
- Identify low-dimensional manifolds
 - ❖ hidden (latent) variables with lower dimensions
- Generate examples from $p(X)$

$$p(x) = \sum_h p(x|h)p(h)$$

- Lack of direct measure of success

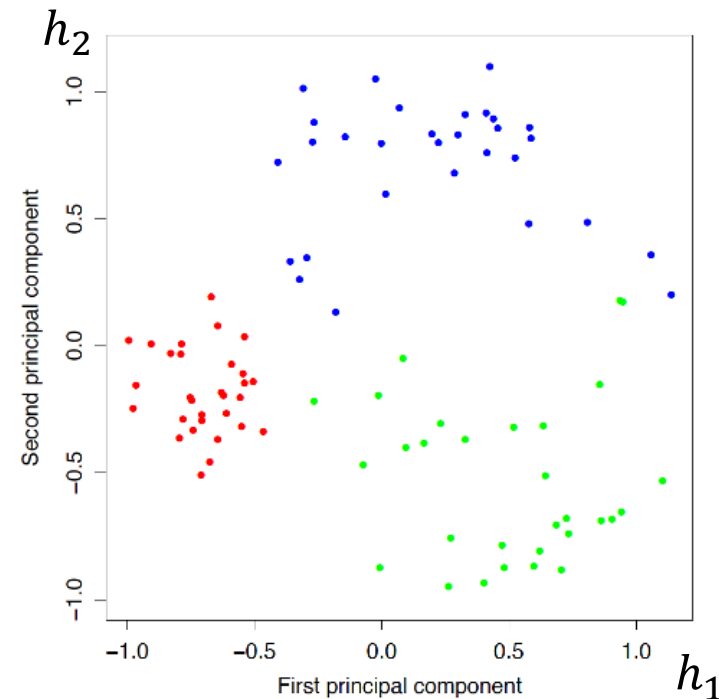
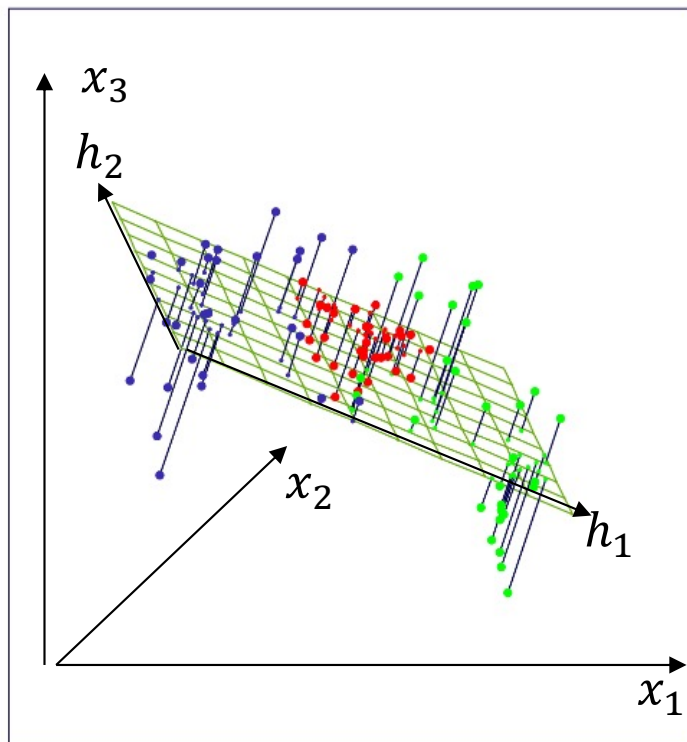
- Use heuristic arguments

Manifold



Principal Component Analysis (PCA)

- ❑ Use $K \leq D$ components to approximate data with D components
- ❑ Often used to compress data dimensions for supervised learning



Linear PCA

□ Linear approximation

$$\hat{x} = f(h) = \mu + V_K h \rightarrow x$$

- V_K is a $D \times K$ matrix with K orthogonal unit vectors as columns

□ Least squares

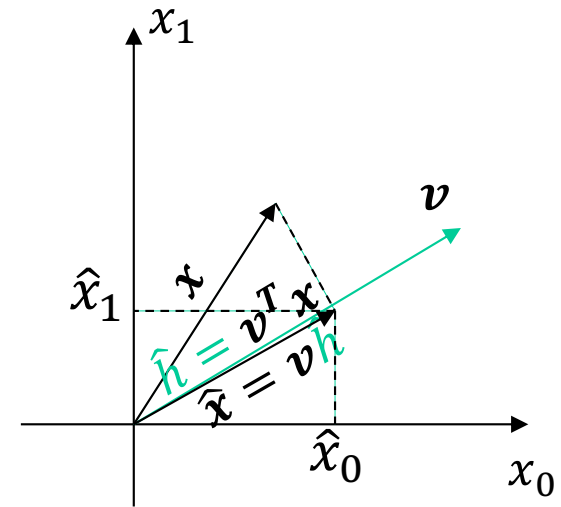
$$\min_{\mu, \{h^{(i)}\}, V_K} \sum_{i=1}^M \|x^{(i)} - \mu - V_K h^{(i)}\|^2$$

□ Minimize w.r.t. each parameter

$$\hat{\mu} = \bar{x}, \quad \hat{h}^{(i)} = V_K^T (x^{(i)} - \bar{x})$$

- We have

$$\min_{V_K} \sum_{i=1}^M \|x_i - \bar{x} - V_K V_K^T (x_i - \bar{x})\|^2$$



□ To find V_K , use singular value decomposition of X

$$X = U \Lambda V^T$$

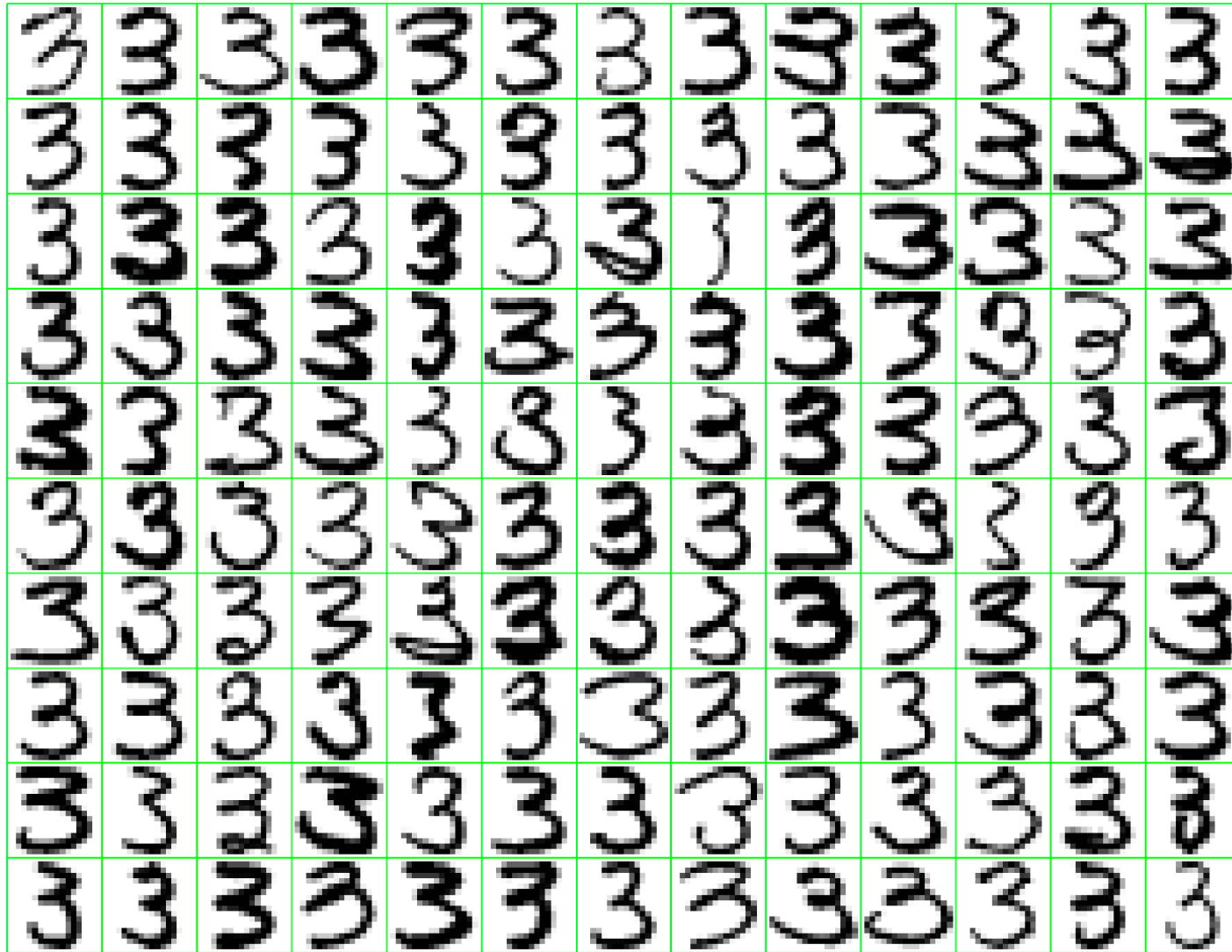
- X is $M \times D$ sample matrix (minus mean), U is $M \times M$ matrix, Λ is $M \times D$ matrix, V is $D \times D$ matrix

□ V_K : first K columns of V with largest singular values

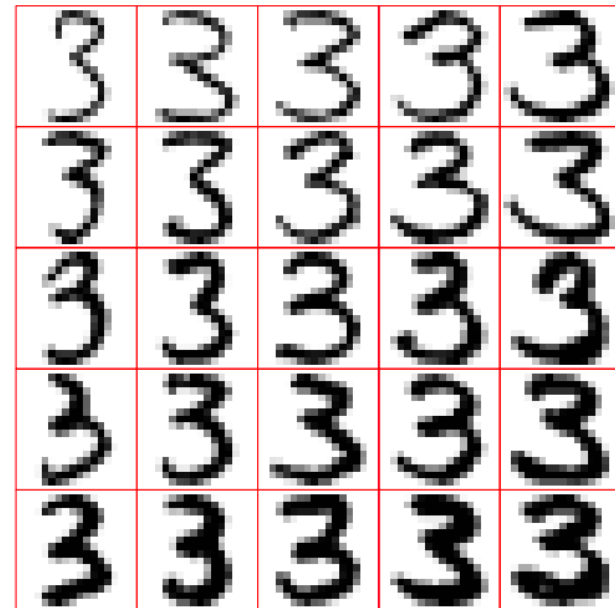
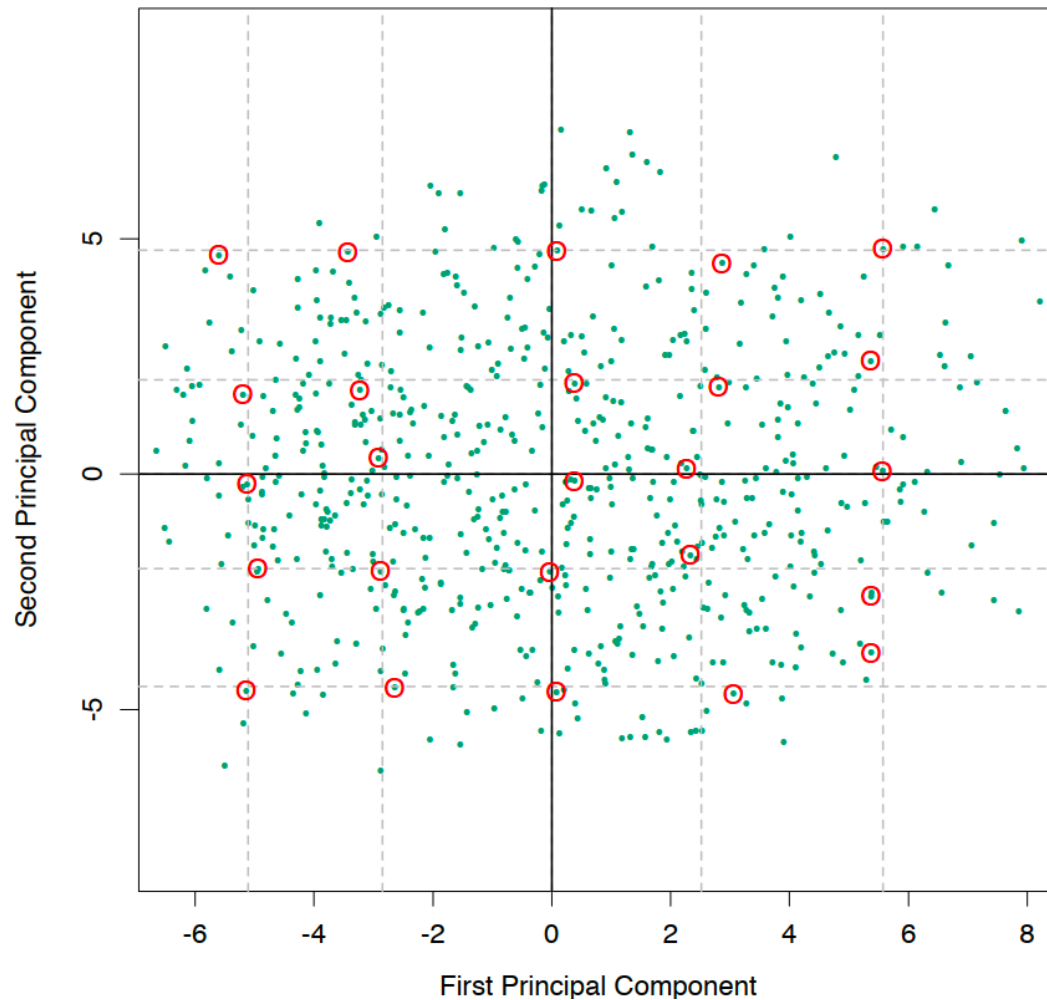
An Interpretation

$$\begin{aligned}\text{Cov}[\mathbf{x}] &= \frac{1}{M-1} X^\top X \\ &= \frac{1}{M-1} (U\Lambda V^\top)^\top (U\Lambda V^\top) \\ &= \frac{1}{M-1} V\Lambda^\top U^\top U\Lambda V^\top \\ &= \frac{1}{M-1} V\Lambda^2 V^\top \\ \text{Cov}[\mathbf{h}] &= \frac{1}{M-1} H^\top H \\ &= \frac{1}{M-1} V^\top X^\top X V \\ &= \frac{1}{M-1} V^\top V\Lambda^2 V^\top V \\ &= \frac{1}{M-1} \Lambda^2\end{aligned}$$

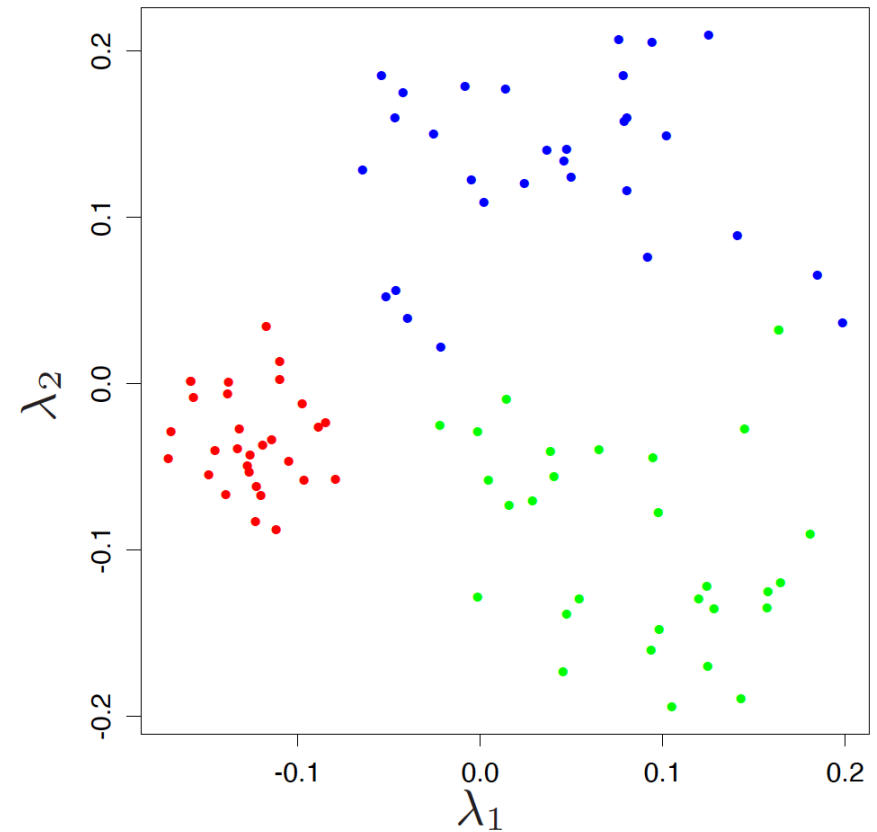
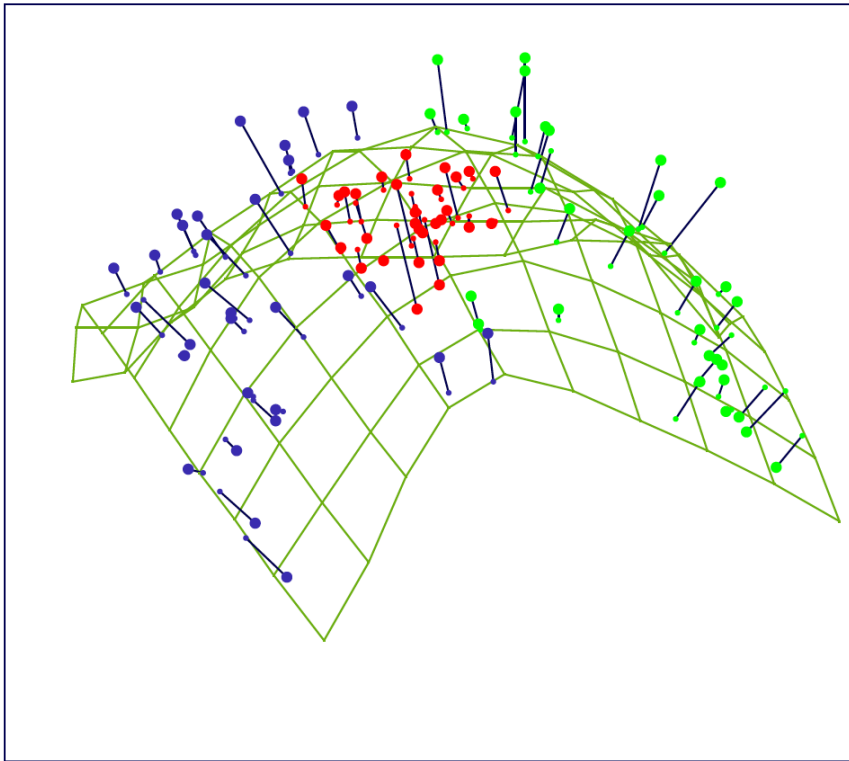
Linear PCA Example



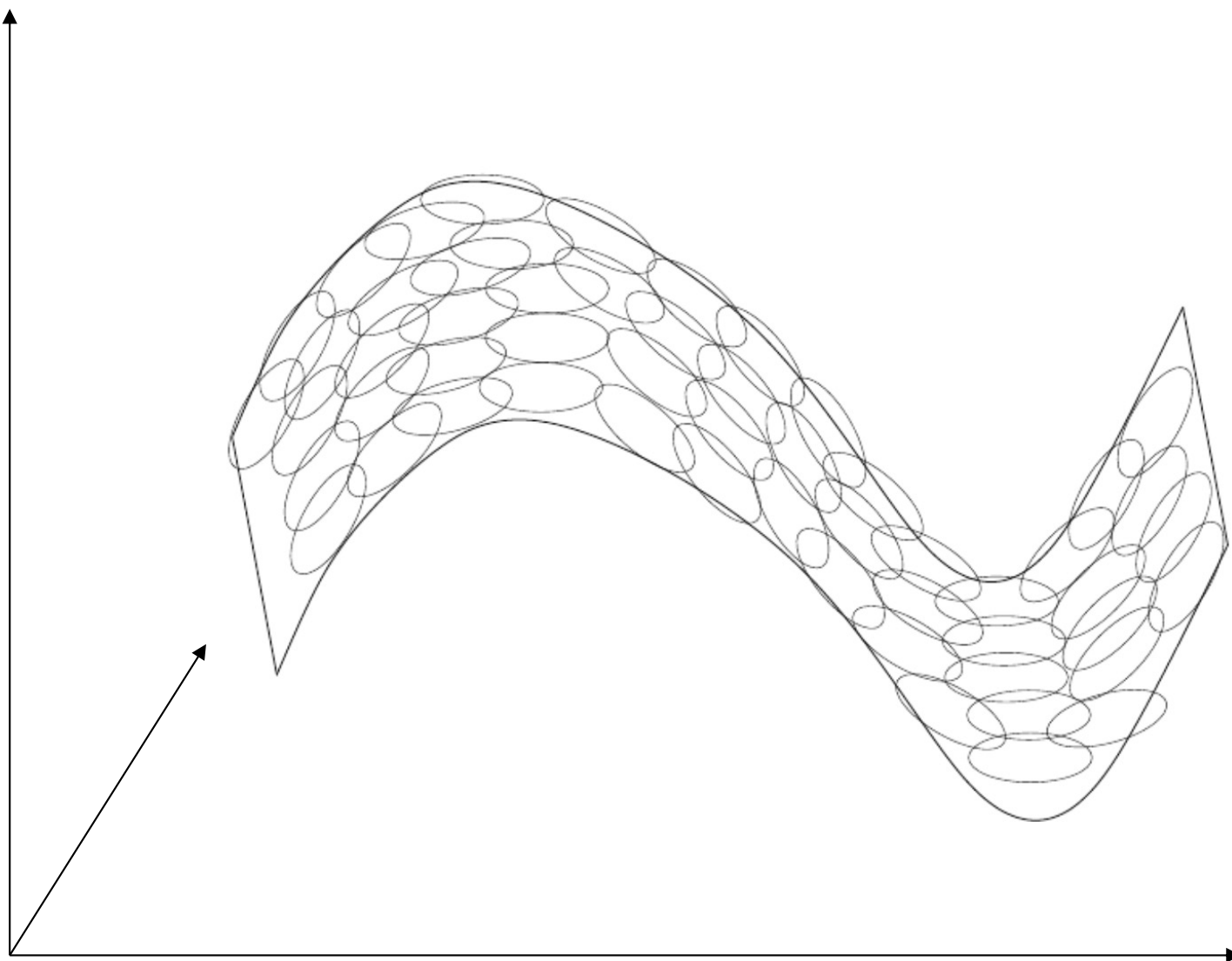
Linear PCA Example



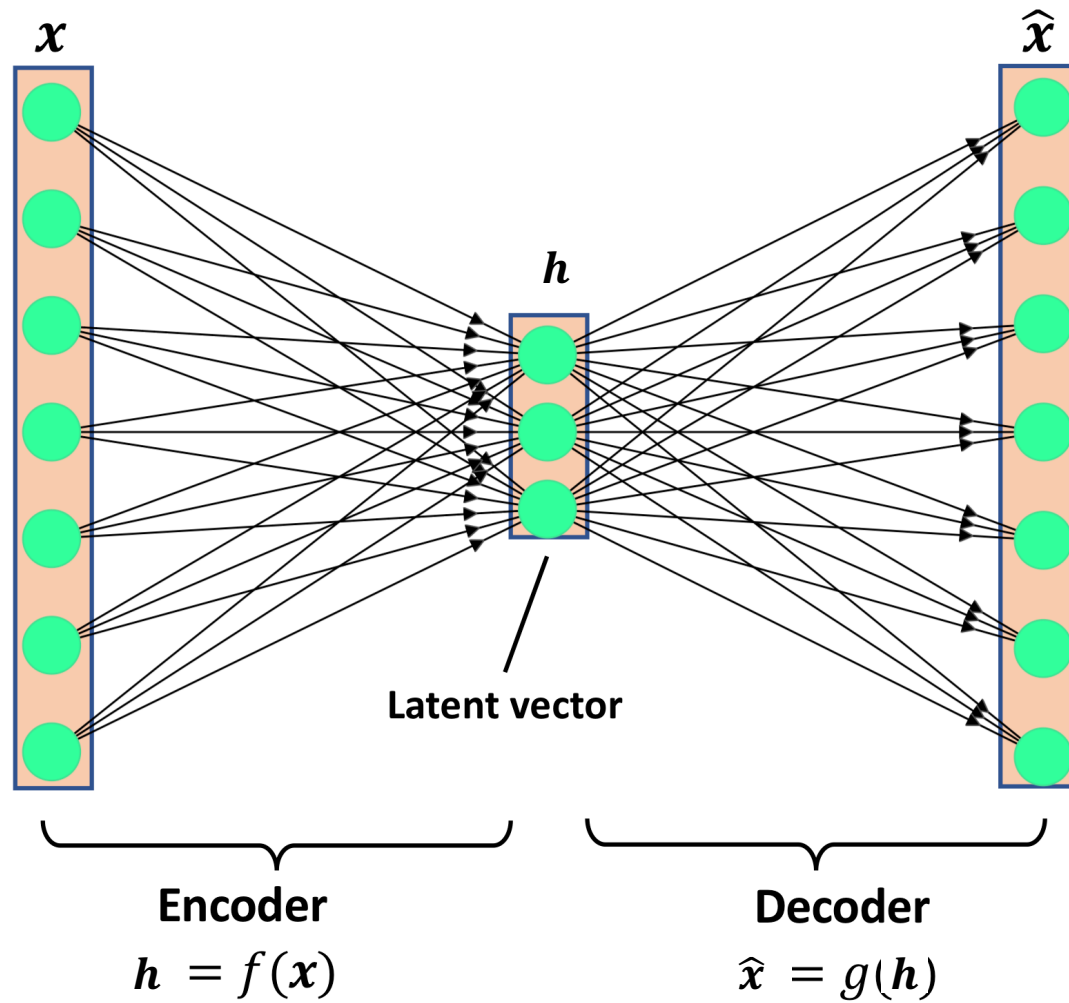
Nonlinear Principal Surface



Manifolds



Autoencoder



Definition

- Motivation: use deep network to represent a larger class

- Definition

- Feedforward Networks

- ❖ Encoder: $H = f_{\theta}(X)$

- ❖ Decoder: $R = g_{\theta'}(H)$

- Stochastic Models

- ❖ Encoder: $p(H|X; \theta)$

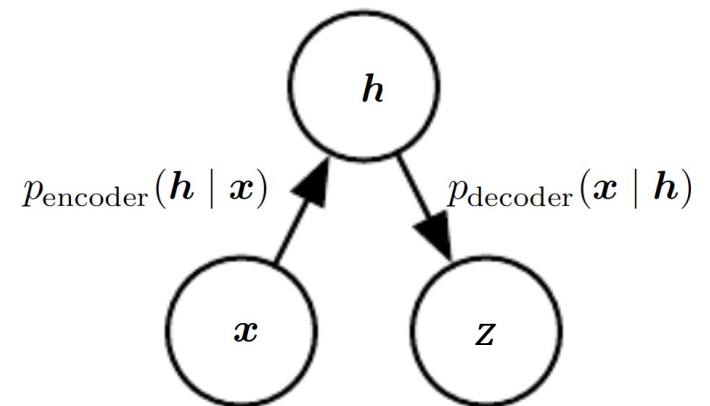
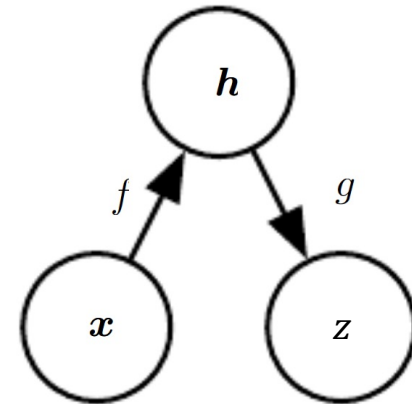
- ❖ Decoder: $p(X|H; \theta')$

- Goal: z similar to x , but not the same

- Undercomplete Autoencoder:
smaller dimension H

$$L(X, g(f(X)))$$

- Overcomplete autoencoder with
regularization for avoiding copying

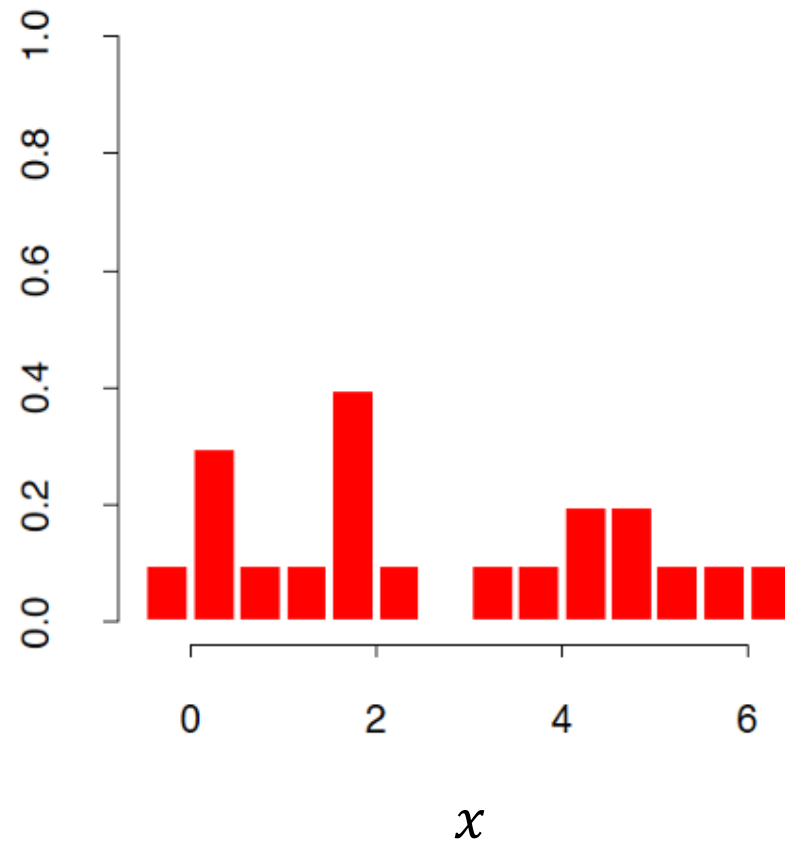


Outline

- Unsupervised Learning Basics
- **Structured Probabilistic Model**
- Training for Undirected Graphical Models
- Restricted Boltzmann Machine

Example

-0.39	0.12	0.94	1.67	1.76	2.44	3.72	4.28	4.92	5.53
0.06	0.48	1.01	1.68	1.80	3.25	4.12	4.60	5.28	6.22



Gaussian Mixture Model

□ A parametric approach

➤ Gaussian mixture model

$$p(x) = \sum_{i=1}^K \alpha_i \phi(x; \mu_i, \Sigma_i), \quad \sum_i \alpha_i = 1$$

□ Consider two-component mixture

$$H \in \{0, 1\}$$

$$X_1 \sim N(\mu_1, \sigma_1^2), \quad X_2 \sim N(\mu_2, \sigma_2^2)$$

$$X = (1 - H)X_1 + HX_2$$

$$\pi = p(H = 1)$$

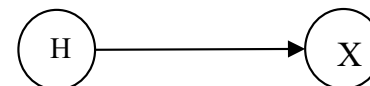
➤ Use $p(x) = \sum_h p(x|h)p(h)$

$$p(x) = (1 - \pi)\phi_{\theta_1}(x) + \pi\phi_{\theta_2}(x),$$

$$\theta = (\pi, \theta_1, \theta_2) = (\pi, \mu_1, \sigma_1^2, \mu_2, \sigma_2^2)$$

$$L(\theta; \mathbf{x}) = \sum_{i=1}^M \log[(1 - \pi)\phi_{\theta_1}(x_i) + \pi\phi_{\theta_2}(x_i)]$$

Hard to maximize



The EM Algorithm

□ If h_i are also observable

$$\begin{aligned} L_0(\theta; \mathbf{x}, \mathbf{h}) &= \sum_{i=1}^M [(1 - h_i) \log[(1 - \pi) \phi_{\theta_1}(x_i)] + h_i \log[\pi \phi_{\theta_2}(x_i)]] \\ &= \sum_{i=1}^M [(1 - h_i) \log \phi_{\theta_1}(x_i) + h_i \log \phi_{\theta_2}(x_i)] + \sum_{i=1}^M [(1 - h_i) \log(1 - \pi) + h_i \log \pi] \end{aligned}$$

MLE

$$\begin{aligned} \hat{\mu}_1 &= \frac{\sum_{i=1}^M (1 - h_i) x_i}{\sum_{i=1}^M (1 - h_i)}, \hat{\sigma}_1^2 = \frac{\sum_{i=1}^M (1 - h_i) (x_i - \hat{\mu}_1)^2}{\sum_{i=1}^M (1 - h_i)} \\ \hat{\mu}_2 &= \frac{\sum_{i=1}^M h_i x_i}{\sum_{i=1}^M h_i}, \hat{\sigma}_2^2 = \frac{\sum_{i=1}^M h_i (x_i - \hat{\mu}_2)^2}{\sum_{i=1}^M h_i}, \hat{\pi} = \frac{\sum_{i=1}^M h_i}{M} \end{aligned}$$

□ h_i are actually hidden

- Guess h_i by $h_i \sim p(h_i | x_i, \theta)$
- Predicting h_i with expected value given x_i, θ
- Then MLE

The EM Algorithm

- Take initial guesses for $\hat{\theta} = (\hat{\pi}, \hat{\theta}_1, \hat{\theta}_2) = (\hat{\pi}, \hat{\mu}_1, \hat{\sigma}_1^2, \hat{\mu}_2, \hat{\sigma}_2^2)$, iterate

- Expectation Step: estimate h_i with

$$\hat{h}_i = \mathbb{E}(h_i | \hat{\theta}, x_i) = p(h_i = 1 | \hat{\theta}, x_i)$$

we know $p(H_i = 0) = (1 - \hat{\pi})$, $p(H_i = 1) = \hat{\pi}$ and $p(x_i | h_i)$

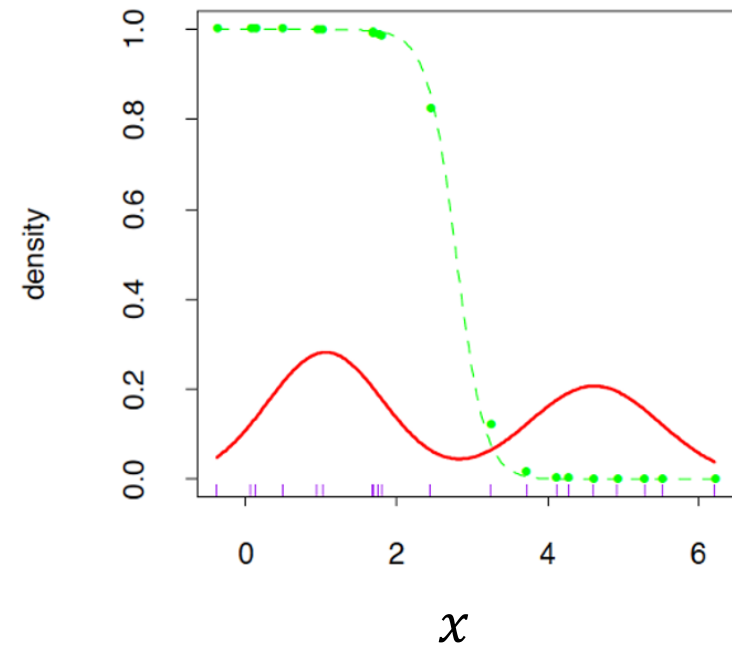
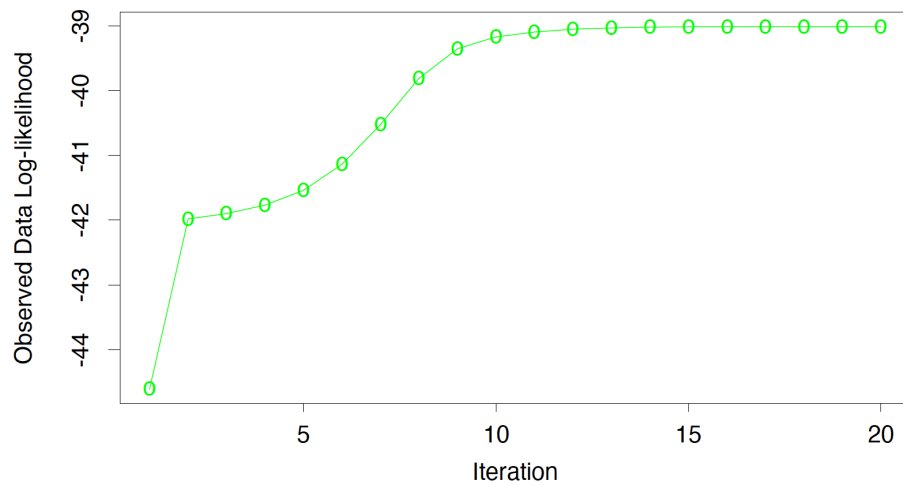
we can estimate responsibility $\hat{h}_i$ with Bayes' formula

$$\hat{h}_i = p(h_i = 1 | \hat{\theta}, x_i) = \frac{\hat{\pi} \phi_{\hat{\theta}_2}(x_i)}{(1 - \hat{\pi}) \phi_{\hat{\theta}_1}(x_i) + \hat{\pi} \phi_{\hat{\theta}_2}(x_i)}$$

- Maximization Step: maximize $L_0(\theta; x, \hat{h})$ to get

$$\hat{\mu}_1 = \frac{\sum_{i=1}^M (1 - \hat{h}_i) x_i}{\sum_{i=1}^M (1 - \hat{h}_i)}, \hat{\sigma}_1^2 = \frac{\sum_{i=1}^M (1 - \hat{h}_i) (x_i - \hat{\mu}_1)^2}{\sum_{i=1}^M (1 - \hat{h}_i)}$$
$$\hat{\mu}_2 = \frac{\sum_{i=1}^M \hat{h}_i x_i}{\sum_{i=1}^M \hat{h}_i}, \hat{\sigma}_2^2 = \frac{\sum_{i=1}^M \hat{h}_i (x_i - \hat{\mu}_2)^2}{\sum_{i=1}^M \hat{h}_i}, \hat{\pi} = \frac{\sum_{i=1}^M \hat{h}_i}{M}$$

Example



Sampling

□ Sampling

- Using chain rule

$$p(\mathbf{x}) = p(x_1) \prod_{i=2}^N p(x_i | x_1, \dots, x_{i-1})$$

- Computing marginal distribution is NP hard

$$p(x_i) = \sum_{x_1} \sum_{x_2} \dots \sum_{x_{i-1}} \sum_{x_{i+1}} \dots \sum_{x_N} p(x_1, x_2, \dots, x_{i-1}, x_i, x_{i+1}, \dots, x_N)$$

- Computing most conditional distributions is still NP hard

$$p(x_i | x_1, \dots, x_{i-1}) = \frac{p(x_1, \dots, x_{i-1}, x_i)}{p(x_1, \dots, x_{i-1})}$$

- Exception

$$p(x_i | x_{-i}) = \frac{p(x_1, \dots, x_N)}{p(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_N)}$$

Markov Chain Monte Carlo (MCMC)

- ❑ Target distribution $p(x)$ is difficult to sample
- ❑ Conditional probabilities may be easier to sample
- ❑ Create a Markov Chain with $p(x)$ as the limiting distribution
- ❑ Process called burning in the Markov Chain
 - Harris chain: generalization to continuous variable

Gibbs Sampling

□ Target

$$p(\mathbf{x})$$

□ Denote

$$p(x_j | \mathbf{x}_{-j}) = p(x_j | x_1, \dots, x_{j-1}, x_{j+1}, \dots, x_N)$$

□ We have

$$p(\mathbf{x}) = p(\mathbf{x}_{-j})p(x_j | \mathbf{x}_{-j}), \forall j$$

Gibbs Sampling

□ Sampling procedure

1. For each $j = 1, \dots, N$, initialize $x_j^{(0)}$; $t = 0$

2. Iterate:

sample $x_1^{(t+1)}$ from $p(x_1|x_{-1}^{(t)})$

sample $x_2^{(t+1)}$ from $p(x_2|x_{-2}^{(t)})$

...

sample $x_N^{(t+1)}$ from $p(x_N|x_{-N}^{(t)})$

$t = t + 1$

Reversibility

□ General Markov chain

$$p(\mathbf{x}^{(t)}, \mathbf{x}^{(t+1)}) = p(\mathbf{x}^{(t)})p(\mathbf{x}^{(t+1)}|\mathbf{x}^{(t)})$$

□ Gibbs sampling

- Select an arbitrary j

$$\begin{aligned} p(\mathbf{x}^{(t)}, \mathbf{x}^{(t+1)}) &= p(\mathbf{x}^{(t)})p(\mathbf{x}^{(t+1)}|\mathbf{x}^{(t)}) \\ &= p(X_j^{(t)} = r, X_{-j}^{(t)} = x_{-j})p(X_j^{(t+1)} = s|X_{-j}^{(t)} = x_{-j}) \\ &= p(X_j^{(t)} = r|X_{-j}^{(t)} = x_{-j})p(X_{-j}^{(t)} = x_{-j})p(X_j^{(t+1)} = s|X_{-j}^{(t)} = x_{-j}) \\ &= p(X_j^{(t)} = r|X_{-j}^{(t-1)} = x_{-j})p(X_{-j}^{(t)} = x_{-j})p(X_j^{(t+1)} = s|X_{-j}^{(t)} = x_{-j}) \\ &= p(X_j = r|x_{-j})p(X_{-j}^{(t)} = x_{-j})p(X_j = s|x_{-j}) \end{aligned}$$



$$p(r)p_{rs} = p(s)p_{sr}$$

- Gibbs sampling is reversible

$$p(\mathbf{x}^{(t)}) \rightarrow p(\mathbf{x})$$

Graph Representation

❑ **Designing** tractable hidden RVs

- Models with complex interactions among RVs
 - ❖ To store $p(X)$ with N dimension taking on k values each, need a table size k^N
 - ❖ Using chain rule is not enough
$$p(x_1, x_2, x_3) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2)$$

❑ Structured probabilistic models

- Factorize distributions into product form with less dimensions
- Modeling only direct interactions between pairwise random variables
 - ❖ An example: $P(x) = \frac{1}{Z} \exp(\sum_i h_i x_i + \sum_{i < j} J_{ij} x_i x_j)$
 - ❖ Some RVs have direct interactions while some don't
 - ❖ Representation in Graphs
 - Node: a random variable
 - Edge: a direct interaction between two RVs $X_i X_j$
- Fewer parameters, less memory, less computing cost

Directed Models

- Belief network or Bayesian network

- Directed graphical models

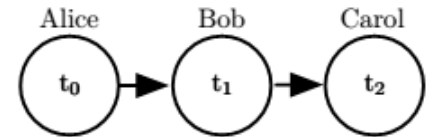
- Directed edge from Node A to Node B :

- A causal relationship: A is a cause for B to happen

$p(B|A)$ is easy to find

- Ex. Alice, Bob, Carol in a relay race

- Conditional independent: Bob is independent from Carol given Alice's finishing time



- Factor distributions into products of conditional distributions

- Regular chain rule:

$$p(t_0, t_1, t_2) = p(t_0)p(t_1|t_0)p(t_2|t_1, t_0)$$

- ❖ Requires 3-d table to implement

- With conditional independence

$$p(t_0, t_1, t_2) = p(t_0)p(t_1|t_0)p(t_2|t_1)$$

- ❖ Requires 2-d table to implement

More on Directed Model

- For any directed acyclic graph $\mathcal{G}$

$$p(X) = \prod_i p(X_i | P(X_i))$$

- $P(X_i)$ represents the parents of X_i
- Applicable to causality flows in one direction

- Complexity

- Assume each node has a small number of parents m
- A table that store conditional probability: $O(k^n) \rightarrow O(k^m)$

- Challenge

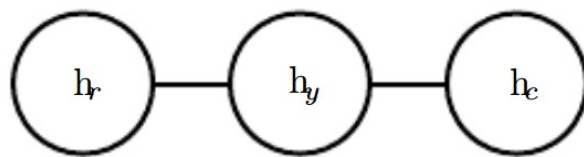
- Reverse direction is complex
 - ❖ Given X_i , its parents are not independent
 - ❖ Need to use Bayesian

Undirected Models

□ Markov Random Fields (MRF) or Markov Networks

➤ An Example

❖ Health of yourself, your colleague, your roommate



□ An undirected link between A and B

- A and B has direct interactions
- No intrinsic direction
- Conditional independent: given h_y , h_r is independent from h_c
- Capture all pairwise direct interactions

Sampling for Structured models

□ Sampling with Directed Models

- Ancestral sampling

$$p(\mathbf{x}) = \prod_i p(x_i | P(x_i))$$

□ Undirected Models

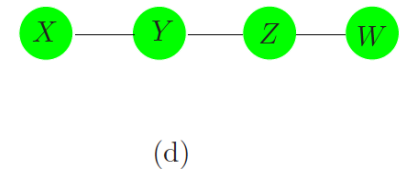
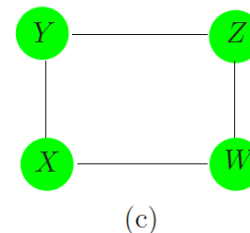
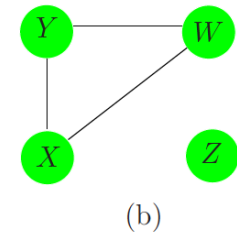
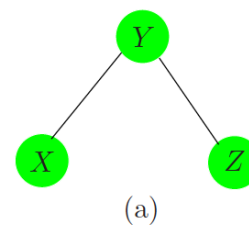
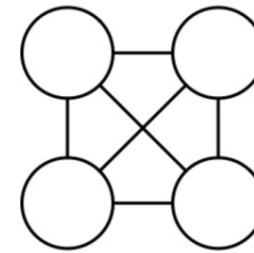
- Gibbs sampling
- Iterative sampling from $p(X_i | X_{-i})$
- Asymptotically converges to correct distribution

□ Mixing time: time to limiting distribution

- Samples correlated
- May be trapped to a mode for multimode distribution
- Depth helps mixing

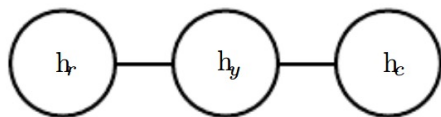
Undirected Models

- Modeling for Markov Network
- Clique: mesh connected subgraph
- A Markov network can be divided into cliques
- Maximal clique: cannot add more edges



More on Undirected Models

- Design probability model for Markov network
- Clique potential $\phi(C)$
 - Measure the affinity of variables in clique C
 - An Example
 - ❖ Health of yourself, your colleague, your roommate



	$h_y = 0$	$h_y = 1$
$h_c = 0$	2	1
$h_c = 1$	1	10

- Unnormalized probability distribution

$$\tilde{p}(X) = \prod_{C \in \mathcal{G}} \phi(C)$$

- Probability function

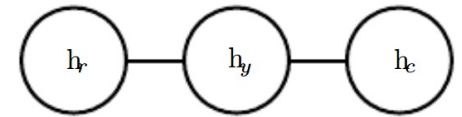
$$p(X) = \frac{1}{Z} \tilde{p}(X)$$

Energy Model

□ Define Energy Probability Model

$$\phi(1) = e^{ah_r + b_1 h_y + w_{ry} h_r h_y}$$

$$\phi(2) = e^{ch_c + b_2 h_y + w_{cy} h_c h_y}$$



$$\begin{aligned} \tilde{p}(\mathbf{h}) &= \prod_{C \in \mathcal{G}} \phi(C) \\ &= e^{ah_r + b_1 h_y + w_{ry} h_r h_y} e^{ch_c + b_2 h_y + w_{cy} h_c h_y} \\ &= e^{ah_r + bh_y + ch_c + w_{ry} h_r h_y + w_{cy} h_c h_y} \end{aligned}$$

$$E(\mathbf{h}) = -(ah_r + bh_y + ch_c + w_{ry} h_r h_y + w_{cy} h_c h_y)$$

$$\tilde{p}(\mathbf{h}) = \exp(-E(\mathbf{h})) \quad p(\mathbf{h}) = \frac{1}{Z} \tilde{p}(\mathbf{h})$$

$$\begin{aligned} p(h_r, h_c | h_y) &= \frac{p(\mathbf{h})}{p(h_y)} = \frac{\tilde{p}(\mathbf{h})}{p(h_y)Z} \\ &= \frac{e^{ah_r + b_1 h_y + w_{ry} h_r h_y} e^{ch_c + b_2 h_y + w_{cy} h_c h_y}}{p(h_y)Z} = \frac{e^{k_r h_r} e^{k_c h_c}}{Z'} \end{aligned}$$

Partition Function

□ Partition function

$$Z = \sum_x \tilde{p}(x)$$

$$Z = \int \tilde{p}(x) dx$$

- Choose ϕ such that Z exists

□ Energy-based models

$$\tilde{p}(X) = \exp(-E(X))$$

- Nice features

- ❖ Capture direct interactions

$$e^{xy}$$

- ❖ Factorize easily with cliques

$$\exp(c_1)\exp(c_2) = \exp(c_1 + c_2)$$

Directed vs Undirected

- ❑ Undirected models capture all second-order direct interactions
 - For modeling symmetric relationships
- ❑ Directed models are special cases of undirected models
 - For modeling causal relationship
 - Only interested in one direction
 - A simplified version
- ❑ Converting a directed to undirected
 - May need to add more links

Outline

- Unsupervised Learning Basics
- Structured Probabilistic Model
- Training for Undirected Graphical Models
- Restricted Boltzmann Machine

Introduction

□ Undirected Models

- A general distribution model

$$p(X; \theta) = \frac{1}{Z(\theta)} \tilde{p}(X; \theta)$$

- Partition function

$$Z(\theta) = \int \tilde{p}(x'; \theta) dx' \text{ or } \sum_x \tilde{p}(x'; \theta)$$

- Operation is intractable for high dimension

- ❖ Find ways to estimate partition function
- ❖ Design models to have tractable partition function
- ❖ Avoid calculating $p(X; \theta)$
 - Get the conditional probabilities without partition function

Gradient of Undirected Models

□ Log-likelihood Gradient

$$\nabla_{\theta} \log p(x; \theta) = \nabla_{\theta} \log \tilde{p}(x; \theta) - \nabla_{\theta} \log Z(\theta)$$

$$\nabla_{\theta} L(\theta) = \sum_{i=1}^M \nabla_{\theta} \log \tilde{p}(x^{(i)}; \theta) - M \nabla_{\theta} \log Z(\theta)$$

➤ Positive phase and negative phase

□ Handling negative phase

$$\begin{aligned} \nabla_{\theta} \log Z(\theta) &= \frac{\nabla_{\theta} Z}{Z} \\ &= \frac{\nabla_{\theta} \sum_{x'} \tilde{p}(x')}{Z} \\ &= \frac{\sum_{x'} \nabla_{\theta} \tilde{p}(x')}{Z} \end{aligned}$$

$\tilde{p}(x') = \exp(\log \tilde{p}(x')) \Rightarrow$

$$\begin{aligned} &\frac{\sum_{x'} \nabla_{\theta} \exp(\log \tilde{p}(x'))}{Z} \\ &= \frac{\sum_{x'} \exp(\log \tilde{p}(x')) \nabla_{\theta} \log \tilde{p}(x')}{Z} \\ &= \frac{\sum_{x'} \tilde{p}(x') \nabla_{\theta} \log \tilde{p}(x')}{Z} \\ &= \sum_{x'} p(x'; \theta) \nabla_{\theta} \log \tilde{p}(x') \\ &= \mathbb{E}_{x' \sim p(x'; \theta)} \nabla_{\theta} \log \tilde{p}(x') \end{aligned}$$

Naïve MCMC

□ Use Gibbs Method to approximate

- Need to generate large number of examples from distribution. Use Gibbs sampling.

Set ϵ , the step size, to a small positive number.

Set k , the number of Gibbs steps, high enough to allow burn in. Perhaps 100 to train an RBM on a small image patch.

while not converged **do**

Sample a minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from the training set

$\mathbf{g} \leftarrow \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} \log \tilde{p}(\mathbf{x}^{(i)}; \boldsymbol{\theta})$.

Initialize a set of m samples $\{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(m)}\}$ to random values (e.g., from a uniform or normal distribution, or possibly a distribution with marginals matched to the model's marginals).

for $i = 1$ to k **do**

for $j = 1$ to m **do**

$\tilde{\mathbf{x}}^{(j)} \leftarrow \text{gibbs_update}(\tilde{\mathbf{x}}^{(j)})$.

end for

end for

$\mathbf{g} \leftarrow \mathbf{g} - \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} \log \tilde{p}(\tilde{\mathbf{x}}^{(i)}; \boldsymbol{\theta})$.

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \mathbf{g}$.

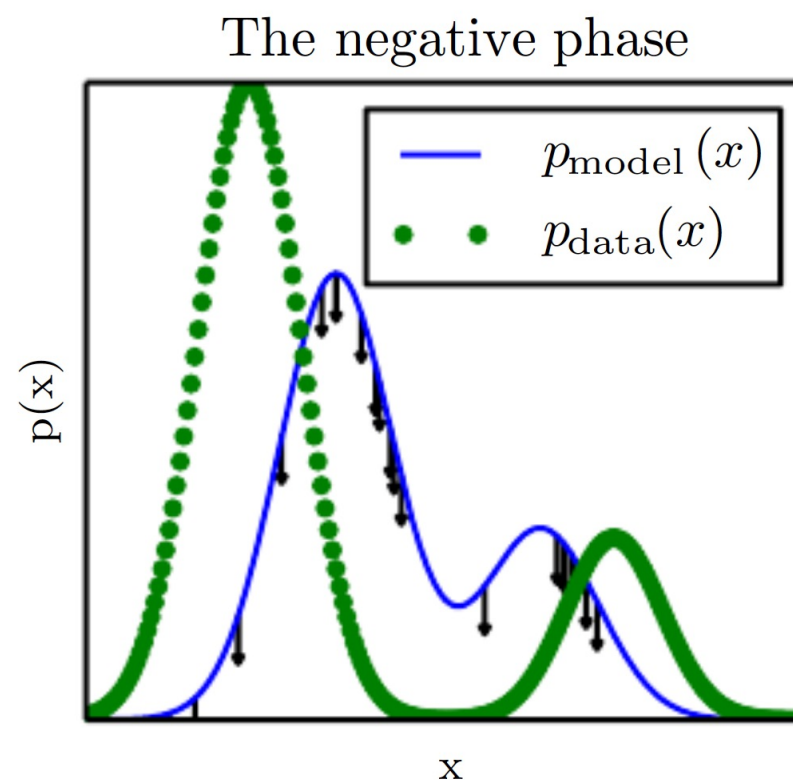
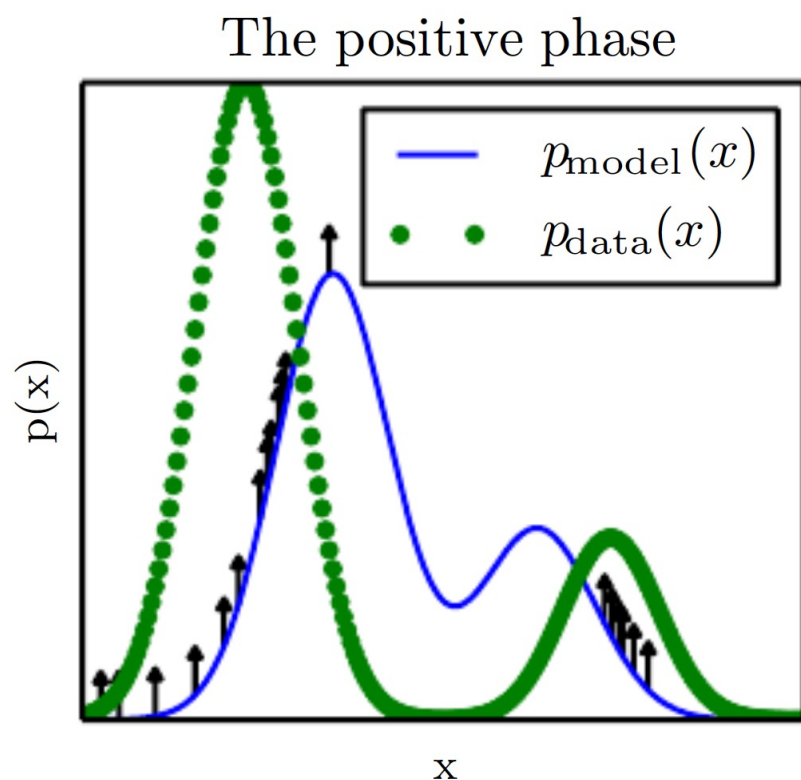
end while

Create m Markov Chains in parallel

□ Burn-in for each gradient step is time consuming

Illustration

- Use Monte Carlo Method to approximate
 - Push up the first part and push down the second part



Outline

- Unsupervised Learning Basics
- Structured Probabilistic Model
- Training for Undirected Graphical Models
- Restricted Boltzmann Machine

Restricted Boltzmann Machines (RBM)

- Use hidden variables to capture the dependency in visible variables
- Restricted Boltzmann Machines (RBM)

$$p(\mathbf{v}, \mathbf{h}) = \frac{1}{Z} \exp(-E(\mathbf{v}, \mathbf{h}))$$

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^T \mathbf{v} - \mathbf{c}^T \mathbf{h} - \mathbf{v}^T \mathbf{W} \mathbf{h}$$

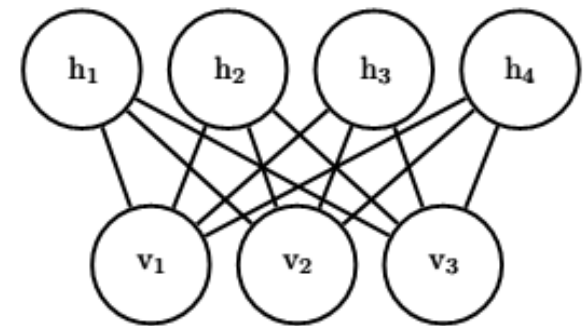
$$Z = \sum_{\mathbf{v}'} \sum_{\mathbf{h}'} \exp\{-E(\mathbf{v}', \mathbf{h}')\}$$

- Bipartite graph
- No intralayer connection
- $p(\mathbf{h}|\mathbf{v})$ and $p(\mathbf{v}|\mathbf{h})$ are factorial

$$p(\mathbf{h}|\mathbf{v}) = \prod_i p(h_i|\mathbf{v})$$

$$p(\mathbf{v}|\mathbf{h}) = \prod_i p(v_i|\mathbf{h})$$

- H_i 's are not independent



Calculate Conditional Distributions

□ Conditional Distributions

$$\begin{aligned} p(\mathbf{h}|\mathbf{v}) &= \frac{p(\mathbf{h},\mathbf{v})}{p(\mathbf{v})} = \frac{1}{p(\mathbf{v})} \frac{1}{Z} \exp\{\mathbf{b}^T \mathbf{v} + \mathbf{c}^T \mathbf{h} + \mathbf{v}^T W \mathbf{h}\} = \frac{1}{Z'} \exp\{\mathbf{c}^T \mathbf{h} + \mathbf{v}^T W \mathbf{h}\} \\ &= \frac{1}{Z'} \exp\{\sum_{j=1}^{N_H} c_j h_j + \sum_{j=1}^{N_H} \mathbf{v}^T W_{:,j} h_j\} \\ &= \frac{1}{Z'} \prod_{j=1}^{N_H} \exp\{c_j h_j + \mathbf{v}^T W_{:,j} h_j\} \\ &= \prod_{j=1}^{N_H} \frac{1}{Z'_j} \exp\{c_j h_j + \mathbf{v}^T W_{:,j} h_j\} = \prod_{j=1}^{N_H} p(h_j|\mathbf{v}) \end{aligned}$$

□ For binary variables

Similar to
logistic regression

$$p(h_j = 1|\mathbf{v}) = \frac{\tilde{p}(h_j = 1|\mathbf{v})/z'_j}{\tilde{p}(h_j = 0|\mathbf{v})/z'_j + \tilde{p}(h_j = 1|\mathbf{v})/z'_j} = \frac{\exp\{c_j + \mathbf{v}^T W_{:,j}\}}{\exp(0) + \exp\{c_j + \mathbf{v}^T W_{:,j}\}} = \sigma(c_j + \mathbf{v}^T W_{:,j})$$

$$p(\mathbf{h}|\mathbf{v}) = \prod_{j=1}^{N_H} \sigma((2\mathbf{h} - 1) \odot (\mathbf{c} + W^T \mathbf{v}))_j$$

$$p(\mathbf{v}|\mathbf{h}) = \prod_{i=1}^{N_V} \sigma((2\mathbf{v} - 1) \odot (\mathbf{b} + W \mathbf{h}))_i$$

□ Marginal distribution

$$p(\mathbf{v}; \theta) = \sum_{\mathbf{h}} p(\mathbf{v}, \mathbf{h}) = \frac{1}{Z} \prod_{i=1}^{N_V} e^{b_i v_i} \prod_{j=1}^{N_H} (1 + e^{c_j + \mathbf{v}^T W_{:,j}})$$

RBM Training

□ The Gradient

$$\begin{aligned}\frac{\partial \log p(\mathbf{v})}{\partial \theta} &= \frac{\partial \log \sum_{\mathbf{h}} p(\mathbf{v}, \mathbf{h})}{\partial \theta} \\ &= - \sum_{\mathbf{h}} p(\mathbf{h}|\mathbf{v}) \frac{\partial E(\mathbf{v}, \mathbf{h})}{\partial \theta} + \sum_{\mathbf{v}', \mathbf{h}'} p(\mathbf{v}', \mathbf{h}') \frac{\partial E(\mathbf{v}', \mathbf{h}')}{\partial \theta} \\ &= -\mathbb{E}_{\theta} \left(\frac{\partial E(\mathbf{v}, \mathbf{h})}{\partial \theta} \middle| \mathbf{v} \right) + \mathbb{E}_{\theta} \left(\frac{\partial E(\mathbf{v}', \mathbf{h}')}{\partial \theta} \right)\end{aligned}$$

Inference

Given $E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^T \mathbf{v} - \mathbf{c}^T \mathbf{h} - \mathbf{v}^T \mathbf{W} \mathbf{h}$, we have

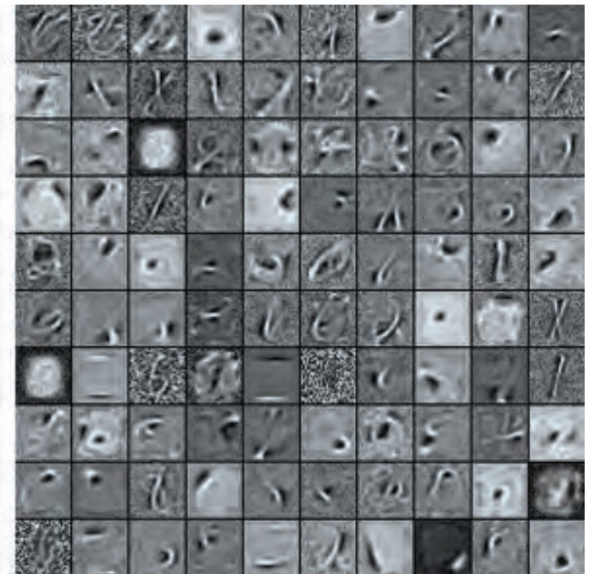
$$\frac{\partial E(\mathbf{v}, \mathbf{h})}{\partial w_{ij}} = -v_i h_j, \quad \frac{\partial E(\mathbf{v}, \mathbf{h})}{\partial b_i} = -v_i, \quad \frac{\partial E(\mathbf{v}, \mathbf{h})}{\partial c_j} = -h_j$$

Then we have

$$\begin{aligned}\frac{\partial \log p(\mathbf{v})}{\partial w_{ij}} &= \mathbb{E}_{\theta}(v_i h_j | \mathbf{v}) - \mathbb{E}_{\theta}(v_i' h_j') \\ &= \sigma(c_j + v^T \mathbf{W}_{:,j}) v_i - \mathbb{E}_{\theta}(v_i' h_j') \\ \frac{\partial \log L(w_{ij})}{\partial w_{ij}} &= \sum_{l=1}^M \sigma(c_j + v^{(l)T} \mathbf{W}_{:,j}) v_i^{(l)} - M \mathbb{E}_{\theta}(v_i' h_j') \\ &= \sum_{l=1}^M \sigma(c_j + v^{(l)T} \mathbf{W}_{:,j}) v_i^{(l)} - \sum_{l=1}^M v_i^{(l)'} h_j^{(l)'}\end{aligned}$$

□ Efficient block Gibbs sampling for partition function

Example



Summary

□ Key concepts

Unsupervised learning, Manifold Learning, Directed Models, Undirected Models, Explaining Away Effect, Clique

□ Models

PCA, Gaussian Mixture, RBM

□ Algorithms

EM Algorithm, MCMC, Ancestral Sampling, Naïve MCMC, CD, SML

References

- ❑ *Deep Learning*, I. Goodfellow, Y. Bengio, and A. Courville, MIT Press, 2016
- ❑ *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, T. Hastie, R. Tibshirani, and J. Friedman, Springer, Second Edition, 2008
- ❑ *Python Machine Learning*, S. Raschka and V. Mirjalili, Packt Publishing, 3rd Edition, 2019